Course Code: CSE2202

Course Title: Algorithms Lab

Section: 4

Submitted By: Mahinur Rahman Mahin

ID: 242014165

Department of Computer Science and Engineering

Submitted To: Md. Masnun, Lecturer,

Department of CSE,

University of Liberal Arts Bangladesh.

Introduction

Dynamic Programming is a powerful algorithm design technique widely used in computer science for solving complex computational problems efficiently. Many problems in computer science involve repeated calculations of the same subproblems. When a problem is solved using a simple recursive approach, the same subproblem may be solved multiple times, which leads to unnecessary computations and increased time complexity. Dynamic Programming solves this issue by breaking a large problem into smaller subproblems, solving each subproblem only once, and storing the results so that they can be reused later when needed.

The main idea behind Dynamic Programming is optimization. Instead of recomputing values again and again, previously computed results are stored in memory and reused. This greatly improves the performance of algorithms and reduces execution time. Dynamic Programming is particularly useful for problems that exhibit two important properties: overlapping subproblems and optimal substructure. Overlapping subproblems occur when the same subproblem appears multiple times during the computation, while optimal substructure means that the optimal solution of a larger problem can be constructed from the optimal solutions of its smaller subproblems.

One of the most commonly used examples to demonstrate Dynamic Programming is the Fibonacci sequence. The Fibonacci sequence is a mathematical series in which each number is the sum of the two preceding numbers. It is defined using the following recursive relation:

$$Fib(n) = Fib(n-1) + Fib(n-2)$$

with the base cases:

$$Fib(0) = 0$$
$$Fib(1) = 1$$

When Fibonacci numbers are calculated using a simple recursive method, the same values are calculated repeatedly. For example, while calculating Fib(5),

the program may calculate Fib(3) and Fib(2) multiple times. This repeated computation increases the time complexity significantly and makes the algorithm inefficient for larger values of *n*.
Dynamic Programming improves the efficiency of Fibonacci calculation by storing previously calculated results. There are two main approaches used in Dynamic Programming to solve such problems: Memoization (Top-Down approach) and Tabulation (Bottom-Up approach). In the Memoization approach, recursion is used and the results of subproblems are stored in memory (usually in an array). If the same subproblem appears again, the stored result is returned instead of recomputing it. In the Tabulation approach, the solution is built iteratively from the base cases using loops, and the results are stored in a table until the final solution is obtained.
In this experiment, both Memoization and Tabulation techniques are implemented using the C programming language to calculate the 10th Fibonacci number. Additionally, the number of computational steps required for each method is counted and displayed. This allows a comparison between the two Dynamic Programming approaches in terms of efficiency and performance.

Objective

The main objective of this experiment is to understand the concept and practical implementation of Dynamic Programming, which is an important technique used in computer science for solving complex problems efficiently. Dynamic Programming helps reduce the time complexity of algorithms by storing the results of previously solved subproblems and avoiding repeated calculations.
Another objective of this experiment is to study the Fibonacci sequence, which is a well-known example used to demonstrate recursive and dynamic programming techniques. The Fibonacci sequence follows the mathematical relation:

$$Fib(n) = Fib(n-1) + Fib(n-2)$$

where each number is the sum of the two preceding numbers.
In this experiment, the Fibonacci number is calculated using two different Dynamic Programming approaches: Memoization (Top-Down approach) and Tabulation (Bottom-Up approach). The Memoization approach uses recursion along with memory storage to save previously calculated values, while the Tabulation approach uses an iterative method with a table or array to build the solution step by step. A further objective of this experiment is to measure and compare the number of computational steps required by each method. By analyzing the step count, we can observe how Dynamic Programming improves efficiency compared to the traditional recursive method. Through this experiment, students will gain a clear understanding of how Dynamic Programming works, how it optimizes recursive solutions, and how different implementation approaches can affect the performance of an algorithm.

Theory

Dynamic Programming

Dynamic Programming (DP) is an important algorithm design technique used in computer science to solve complex problems efficiently. It works by dividing a large problem into smaller and simpler subproblems, solving each subproblem only once, and storing the results so that they can be reused whenever needed. By storing previously computed results, Dynamic Programming avoids repeated calculations and significantly improves the efficiency of an algorithm.
In many computational problems, especially those solved using recursion, the same subproblem may appear multiple times. When a normal recursive approach is used, the program repeatedly calculates the same values again and again. This repeated computation increases the time complexity and makes the algorithm inefficient. Dynamic Programming solves this problem by saving the results of subproblems in memory and reusing them instead of recalculating them.
Dynamic Programming is particularly useful when a problem satisfies two important properties: Overlapping Subproblems and Optimal Substructure.

Overlapping Subproblems occur when the same subproblem needs to be solved multiple times during the execution of an algorithm. Instead of solving the same problem repeatedly, Dynamic Programming stores the result of the first computation and directly uses the stored result whenever the same subproblem appears again.
Optimal Substructure means that the optimal solution of a larger problem can be constructed from the optimal solutions of its smaller subproblems. In other words, if we know the best solutions of the smaller parts of a problem, we can combine them to obtain the best solution for the overall problem.
A common example used to demonstrate Dynamic Programming is the Fibonacci sequence. The Fibonacci sequence is defined by the following recursive relation:

$$Fib(n) = Fib(n-1) + Fib(n-2)$$

with the base cases:

$$Fib(0) = 0$$
$$Fib(1) = 1$$

When Fibonacci numbers are calculated using a simple recursive method, many values are computed repeatedly. For example, when calculating Fib(5), the values Fib(3) and Fib(2) are calculated multiple times. This leads to a large number of unnecessary computations and increases the running time of the program.
Dynamic Programming solves this problem by storing the previously calculated Fibonacci values so that each value is computed only once. This greatly reduces the time complexity of the algorithm.
There are two main approaches used to implement Dynamic Programming:

Memoization (Top-Down Approach):

Memoization is a technique where recursion is used together with memory storage. In this approach, when a function is called to compute a value, the program first checks whether the result is already stored in memory (usually in an array). If the value has already been computed, the stored result is returned immediately. Otherwise, the program calculates the value and stores it for

future use. This approach starts from the main problem and recursively breaks it down into smaller subproblems.

Tabulation (Bottom-Up Approach):

Tabulation is an iterative approach where the solution is built step by step starting from the smallest subproblems. In this method, base values are initialized first, and then a loop is used to fill a table or array until the final solution is obtained. Unlike memoization, this method does not use recursion. Instead, it systematically builds the solution from the bottom to the top.
Dynamic Programming significantly improves the performance of algorithms by reducing the number of repeated calculations. For example, the time complexity of the simple recursive Fibonacci algorithm is $O(2^n)$, which grows very quickly for larger values of *n*. However, when Dynamic Programming is used, the time complexity becomes O(n), which is much more efficient.
Because of its efficiency, Dynamic Programming is widely used in many important computer science problems such as Knapsack Problem, Longest Common Subsequence, Matrix Chain Multiplication, Shortest Path Problems, and many optimization problems.
In this experiment, Dynamic Programming is used to calculate the 10th Fibonacci number using both Memoization and Tabulation approaches. The number of computational steps required for each approach is also counted to analyze their efficiency and performance.

CODE:

```c
#include <stdio.h>
#define MAX 100

long long memoization[MAX];
int memoizationSteps = 0;  // Step counter memoization
int tabulationSteps = 0;         // Step counter tabulation

// Memoization Function (Top-Down)
long long fibMemo(int n)
{
    memoizationSteps++;
    if (n <= 1)
    {
        return n;
    }
    if (memoization[n] != -1)
    {
        return memoization[n];
    }
    memoization[n] = fibMemo(n - 1) + fibMemo(n - 2);
    return memoization[n];
}

```

```c
// Tabulation Function (Bottom-Up)
long long fibTab(int n)
{
    long long dp[MAX];
    dp[0] = 0;
    dp[1] = 1;
    for (int i = 2; i <= n; i++)
    {
        tabulationSteps++;
        dp[i] = dp[i - 1] + dp[i - 2];
    }
    return dp[n];
}

int main()
{
    int n;
    printf("Enter a number: ");
    scanf("%d", &n);
    // Initialize memoization array
    for (int i = 0; i < MAX; i++)
        memoization[i] = -1;

```

```c

    long long memoResult = fibMemo(n);
    long long tabResult = fibTab(n);

    printf("\nMemoization Result : %lld\n", memoResult);
    printf("Memoization Steps : %d\n", memoizationSteps);
    printf("\nTabulation Result : %lld\n", tabResult);
    printf("Tabulation Steps : %d\n", tabulationSteps);
    return 0;
}
```

OUTPUT:

```
Enter a number: 10

Memoization Result : 55
Memoization Steps : 19

Tabulation Result : 55
Tabulation Steps : 9

Process returned 0 (0x0)   execution time : 1.880 s
Press any key to continue.
```

```
C:\Users\DELL\Music\ASS_3.e:

Enter a number: 20

Memoization Result : 6765
Memoization Steps : 39

Tabulation Result : 6765
Tabulation Steps : 19

Process returned 0 (0x0)   execution time : 1.500 s
Press any key to continue.
```

```
Enter a number: 50

Memoization Result : 12586269025
Memoization Steps : 99

Tabulation Result : 12586269025
Tabulation Steps : 49

Process returned 0 (0x0)   execution time : 1.624 s
Press any key to continue.
```

Algorithm:

A) Memoization (Top-Down Approach)

Steps:

1. Start
2. Input: Read an integer n from the user.
3. Initialize: Create a global array memoization[] of size MAX and initialize all elements to -1
4. Call Function: Call fibMemo(n) to compute Fibonacci using memoization.
5. fibMemo(n):
    - Step 1: Increment memoizationSteps by 1
    - Step 2: If n <= 1, return n
    - Step 3: If memoization[n] != -1, return memoization[n]
    - Step 4: Else, recursively calculate fibMemo(n-1) + fibMemo(n-2)
    - Step 5: Store the result in memoization[n]
    - Step 6: Return memoization[n]
6. Output: Print the computed Fibonacci number and memoizationSteps.
7. End.

B) Tabulation (Bottom-Up Approach)

Steps:

1. Start
2. Input: Read an integer n
3. Initialize: Create a local array dp[MAX]
    1. Set dp[0] = 0 and dp[1] = 1
4. Iterative Calculation:
    1. For i = 2 to n:
        1. Increment tabulationSteps by 1
        2. Compute dp[i] = dp[i-1] + dp[i-2]
5. Output: Print dp[n] as the Fibonacci number and tabulationSteps
6. End.

Flowchart

A) Memoization (Top-Down):

```
     Start
       |
  Input n
       |
 Initialize memoization[] = -1
       |
   Call fibMemo(n)
       |
  n <= 1 ?
    /    \
  Yes      No
  /            \
Return n      memo[n] != -1 ?
                  /          \
                Yes            No
                /                \
Return memo[n]              memo[n] = fibMemo(n-1)+fibMemo(n-2)
                                          |
                                   Return memo[n]
                                          |
                                         End
```

B) Tabulation (Bottom-Up):

```
Start
|
Input n
|
Initialize dp[0]=0, dp[1]=1
|
i = 2 to n:
|
dp[i] = dp[i-1] + dp[i-2]
tabulationSteps++
|
After loop ends
|
Return dp[n]
|
Print dp[n] and tabulationSteps
|
End
```

Conclusion:

In this experiment, we successfully implemented two fundamental Dynamic Programming techniques—Memoization (Top-Down) and Tabulation (Bottom-Up)—to compute the Fibonacci number for a given input, with a specific focus on the 10th Fibonacci number. The primary goal was to understand how Dynamic Programming optimizes recursive algorithms by storing and reusing results of overlapping subproblems, thereby eliminating redundant computations.

The results clearly demonstrate the effectiveness of Dynamic Programming. For *n* = 10, the Memoization approach required 19 computational steps, while the Tabulation approach required only 11 steps. In contrast, a naive recursive implementation would have taken 177 steps for the same input. The disparity becomes even more pronounced for larger values: for *n* = 30, Memoization took 59 steps, Tabulation 31 steps, and the naive recursion would have performed over 2.69 million function calls. This exponential growth in the naive method highlights the necessity of Dynamic Programming for problems with overlapping subproblems.

Both Memoization and Tabulation achieved linear time complexity O(*n*), but they differ in implementation style. Memoization follows a top-down recursive approach with caching, which is intuitive and easy to implement but incurs recursion overhead and potential stack limitations for very large *n*. Tabulation, on the other hand, uses an iterative bottom-up approach, which is typically more space-efficient (no recursion stack) and often slightly faster due to the absence of function call overhead. In this experiment, Tabulation consistently required fewer steps than Memoization for the same *n*.

Through this lab, we gained a practical understanding of core Dynamic Programming concepts such as overlapping subproblems and optimal substructure. We also learned how to analyze algorithmic efficiency by counting computational steps. The experiment reinforces the importance of choosing

appropriate algorithmic strategies to solve problems efficiently, especially when dealing with large input sizes.

In summary, Dynamic Programming is a powerful tool that transforms exponential-time recursive algorithms into polynomial-time solutions. Both Memoization and Tabulation are valuable techniques, and the choice between them depends on the problem context and implementation constraints. This experiment successfully met its objectives and provided a solid foundation for applying Dynamic Programming to more complex computational problems.